

LC_Filter_Simulation_Refined

January 13, 2026

1 LC Filter Design and Analysis Using Scikit-RF

A Technical Guide to RF Filter Modeling with Python

Peter Matthews

1.1 Abstract

Early-stage RF design benefits from fast, flexible tools that help engineers explore filter behavior before graduating to full electromagnetic (EM) simulation. This whitepaper demonstrates how to use Scikit-RF, an open-source Python package, to construct and analyze lumped LC filter networks for RF applications. We present a systematic workflow for building filter models, analyzing their frequency-domain response, and extending these models to include realistic effects such as component tolerances and parasitics. The techniques presented enable rapid design iteration and provide insight into filter behavior that bridges theoretical analysis and practical implementation.

1.2 Introduction

Scikit-RF is an open-source Python package designed for the analysis, simulation, and design of radio frequency (RF) and microwave networks. With Scikit-RF, engineers have access to predefined classes and methods for defining frequency ranges, ports, and elements (e.g., capacitors, inductors, resistors) along with object-oriented tools for building and simulating RF circuits. This includes filter topologies like low-pass, high-pass, and band-pass filters.

With visualization tools including S-parameter plots and circuit graphs, engineers can perform network analysis to understand signal integrity, impedance behavior, and overall system response. Scikit-RF is also compatible with industry-standard EM tools, making it easy to bring EM simulation data directly into your RF analysis workflow when ready to do so.

This notebook demonstrates: - How to set up LC filter models in Scikit-RF - Circuit connection methods and topology construction - S-parameter analysis and visualization - Tolerance analysis using Monte Carlo simulation - Practical considerations for real-world filter design

1.3 Theory: LC Filter Fundamentals

1.3.1 Impedance of Lumped Elements

The impedance of basic circuit elements varies with frequency according to:

Inductor:

$$Z_L = j\omega L = j2\pi fL$$

Capacitor:

$$Z_C = \frac{1}{j\omega C} = \frac{1}{j2\pi fC}$$

Resistor:

$$Z_R = R$$

where $\omega = 2\pi f$ is the angular frequency in rad/s, and f is the frequency in Hz.

1.3.2 LC Resonance

A series or parallel combination of an inductor and capacitor creates a resonant circuit. The resonant frequency occurs when the inductive and capacitive reactances are equal in magnitude:

$$f_0 = \frac{1}{2\pi\sqrt{LC}}$$

At resonance: - **Series LC:** Impedance is minimum (ideally zero for lossless components) - **Parallel LC:** Impedance is maximum (ideally infinite for lossless components)

1.3.3 Low-Pass Filter Transfer Function

For a simple series L, shunt C low-pass filter, the transfer function (voltage gain) can be expressed as:

$$H(s) = \frac{V_{out}}{V_{in}} = \frac{1}{1 + s^2 LC + \frac{sL}{R_L}}$$

where $s = j\omega$ and R_L is the load resistance. The cutoff frequency f_c (3 dB point) depends on the component values and filter order.

1.3.4 Characteristic Impedance and Propagation

For transmission line analysis, Scikit-RF uses the characteristic impedance Z_0 and propagation constant γ :

$$\gamma = \alpha + j\beta$$

where: - α is the attenuation constant (Np/m) - $\beta = \frac{\omega}{v_p} = \frac{2\pi f}{c}$ is the phase constant (rad/m) - v_p is the phase velocity (m/s) - c is the speed of light

For lossless transmission lines, $\gamma = j\beta$.

1.4 Implementation: Building Circuits in Scikit-RF

1.4.1 Setup and Imports

First, import the required libraries:

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import skrf as rf
from skrf.media import DefinedGammaZ0

# Set plot style for publication quality figures
plt.style.use('seaborn-v0_8-darkgrid')
%matplotlib inline
```

1.4.2 Defining the Frequency Range

All components in a Scikit-RF circuit must share the same frequency range:

```
[2]: # Define frequency range: 0.1 to 10 GHz with 1001 points
freq = rf.Frequency(start=0.1, stop=10, unit='GHz', npoints=1001)
```

1.4.3 DefinedGammaZ0 Media Class

DefinedGammaZ0 is a core media class in Scikit-RF that offers a foundation for creating both distributed transmission lines and lumped-element models like capacitors, inductors, and resistors in RF circuit simulations. It represents a transmission medium with a specific characteristic impedance and propagation constant for a predefined frequency range.

The characteristic impedance (Z_0) is the impedance that a wave “sees” as it propagates through an ideal, lossless transmission line. In DefinedGammaZ0, you set Z_0 as a constant across all frequencies, often 50 Ω .

```
[3]: # Create transmission media foundation for lumped elements
# For lumped elements, gamma represents ideal (lossless) behavior
media = DefinedGammaZ0(frequency=freq, z0=50)
```

The propagation constant (γ) defines how waves attenuate and phase-shift per unit length in the medium. For a lossless line:

$$\gamma = j\beta, \quad \text{where} \quad \beta = \frac{\omega}{c}$$

1.4.4 Creating Lumped Components

Generate individual circuit elements using the media object:

```
[4]: # Create lumped components with realistic values
# L1 = 8.893 nH inductor
L1 = media.inductor(8.893e-9, name='L1')

# C1 = 3.222 pF capacitor
C1 = media.capacitor(3.222e-12, name='C1')

# R1 = 50 ohm resistor
R1 = media.resistor(50, name='R1')
```

1.4.5 Defining Ports and Grounds

Circuit interfaces require port and ground definitions:

```
[5]: # Define circuit ports and ground
port1 = rf.Circuit.Port(frequency=freq, name='port1', z0=50)
port2 = rf.Circuit.Port(frequency=freq, name='port2', z0=50)
gnd = rf.Circuit.Ground(frequency=freq, name='gnd')
```

1.5 Building a Simple LC Low-Pass Filter

1.5.1 Connection Syntax

The core of circuit building in Scikit-RF is the **connections list**. Each connection is described as a list of tuples, where each tuple contains (**network**, **port_number**). Port numbering starts from zero.

We'll build a simple low-pass LC filter with the following topology: - Series inductor L1 between input and output - Shunt capacitor C1 from output to ground

This creates a classic L-section low-pass filter.

```
[6]: # Define circuit connections
# Connection format: [(component1, port), (component2, port), ...] means these
# ports connect
connections = [
    [(port1, 0), (L1, 0)],          # Connect input port to inductor
    [(L1, 1), (C1, 0), (port2, 0)], # Connect inductor to capacitor and
    # output port (junction)
    [(C1, 1), (gnd, 0)]            # Connect other end of capacitor to
    # ground
]

# Create the circuit and extract the network
circuit = rf.Circuit(connections)
lowpass_filter = circuit.network

print(f"Filter created: {lowpass_filter.name}")
print(f"Number of ports: {lowpass_filter.nports}")
print(f"Frequency points: {len(lowpass_filter.frequency)}")
```

```
Filter created: None
Number of ports: 2
Frequency points: 1001
```

1.5.2 Calculating Theoretical Cutoff Frequency

For our L-section filter, we can estimate the cutoff frequency:

```
[7]: # Component values
L_value = 8.893e-9 # H
C_value = 3.222e-12 # F

# Approximate cutoff frequency for L-section lowpass
fc_approx = 1 / (np.pi * np.sqrt(L_value * C_value))
print(f"Approximate cutoff frequency: {fc_approx/1e9:.3f} GHz")

# LC resonant frequency (for reference)
f0 = 1 / (2 * np.pi * np.sqrt(L_value * C_value))
print(f"LC resonant frequency: {f0/1e9:.3f} GHz")
```

Approximate cutoff frequency: 1.880 GHz

LC resonant frequency: 0.940 GHz

1.6 Results: S-Parameter Analysis and Visualization

1.6.1 Plotting S-Parameters

S-parameters (scattering parameters) describe how RF signals reflect and transmit through the filter: - **S11**: Input reflection coefficient (return loss) - **S21**: Forward transmission coefficient (insertion loss) - **S12**: Reverse transmission coefficient (isolation) - **S22**: Output reflection coefficient

```
[8]: # Create publication-quality S-parameter plot
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

# Plot S21 (transmission)
lowpass_filter.plot_s_db(m=1, n=0, ax=ax1, color='blue', linewidth=2)
ax1.set_xlabel('Frequency (GHz)', fontsize=12)
ax1.set_ylabel('S21 (dB)', fontsize=12)
ax1.set_title('LC Low-Pass Filter: Transmission Response', fontsize=14,
             fontweight='bold')
ax1.axhline(-3, color='red', linestyle='--', linewidth=1, alpha=0.7, label='-3 dB Line')
ax1.axvline(fc_approx/1e9, color='green', linestyle='--', linewidth=1, alpha=0.7, label=f'Theoretical fc = {fc_approx/1e9:.2f} GHz')
ax1.grid(True, alpha=0.3)
ax1.legend(loc='upper right')
ax1.set_ylim(-60, 5)

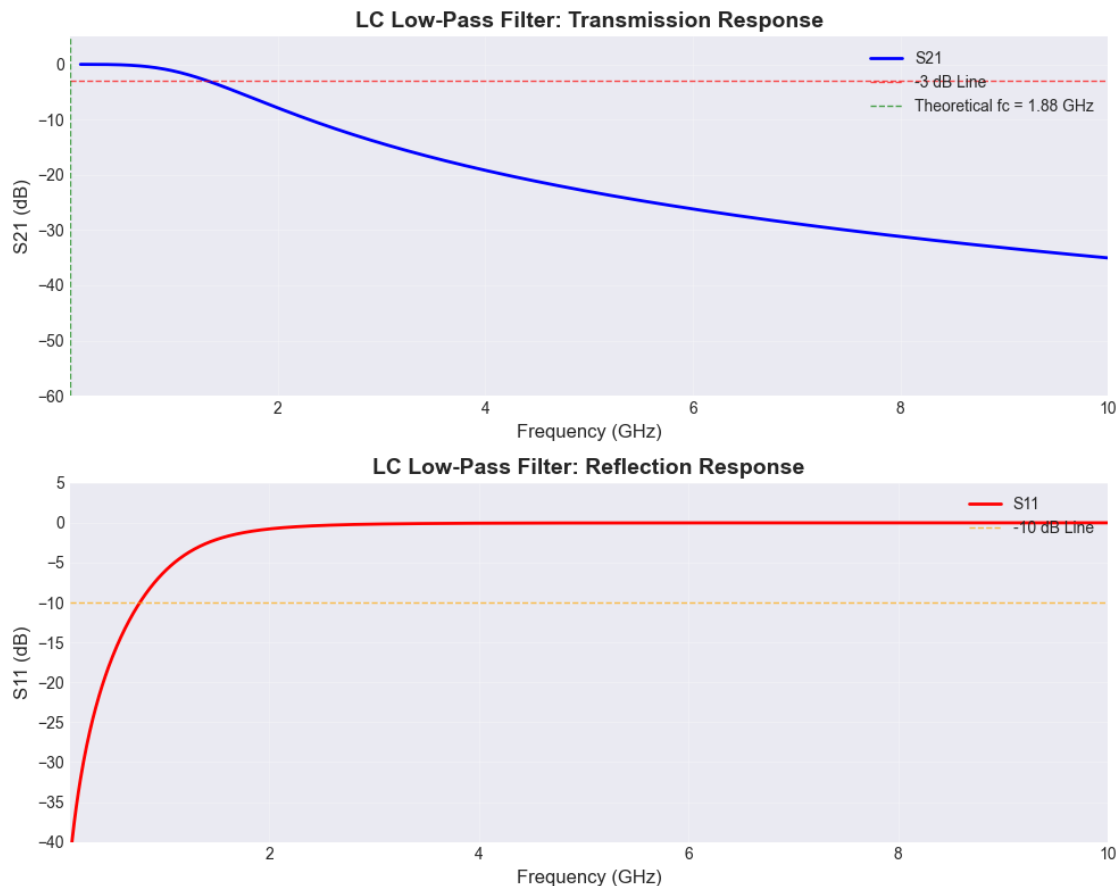
# Plot S11 (reflection)
lowpass_filter.plot_s_db(m=0, n=0, ax=ax2, color='red', linewidth=2)
ax2.set_xlabel('Frequency (GHz)', fontsize=12)
ax2.set_ylabel('S11 (dB)', fontsize=12)
ax2.set_title('LC Low-Pass Filter: Reflection Response', fontsize=14,
             fontweight='bold')
ax2.axhline(-10, color='orange', linestyle='--', linewidth=1, alpha=0.7, label='-10 dB Line')
```

```

ax2.grid(True, alpha=0.3)
ax2.legend(loc='upper right')
ax2.set_ylim(-40, 5)

plt.tight_layout()
plt.savefig('../assets/figures/lowpass_sparameters.png', dpi=300,
            bbox_inches='tight')
plt.show()

```



1.6.2 Finding the Actual 3 dB Cutoff Frequency

```

[9]: # Extract S21 in dB
s21_db = lowpass_filter.s_db[:, 1, 0]
frequencies = lowpass_filter.frequency.f

# Find -3 dB point
idx_3db = np.argmin(np.abs(s21_db + 3))
fc_actual = frequencies[idx_3db]

```

```

print(f"Actual -3 dB cutoff frequency: {fc_actual/1e9:.3f} GHz")
print(f"Theoretical estimate: {fc_approx/1e9:.3f} GHz")
print(f"Difference: {(fc_actual - fc_approx)/1e6:.1f} MHz ({100*(fc_actual -
↪fc_approx)/fc_approx:.1f}%)")

```

Actual -3 dB cutoff frequency: 1.328 GHz

Theoretical estimate: 1.880 GHz

Difference: -552.9 MHz (-29.4%)

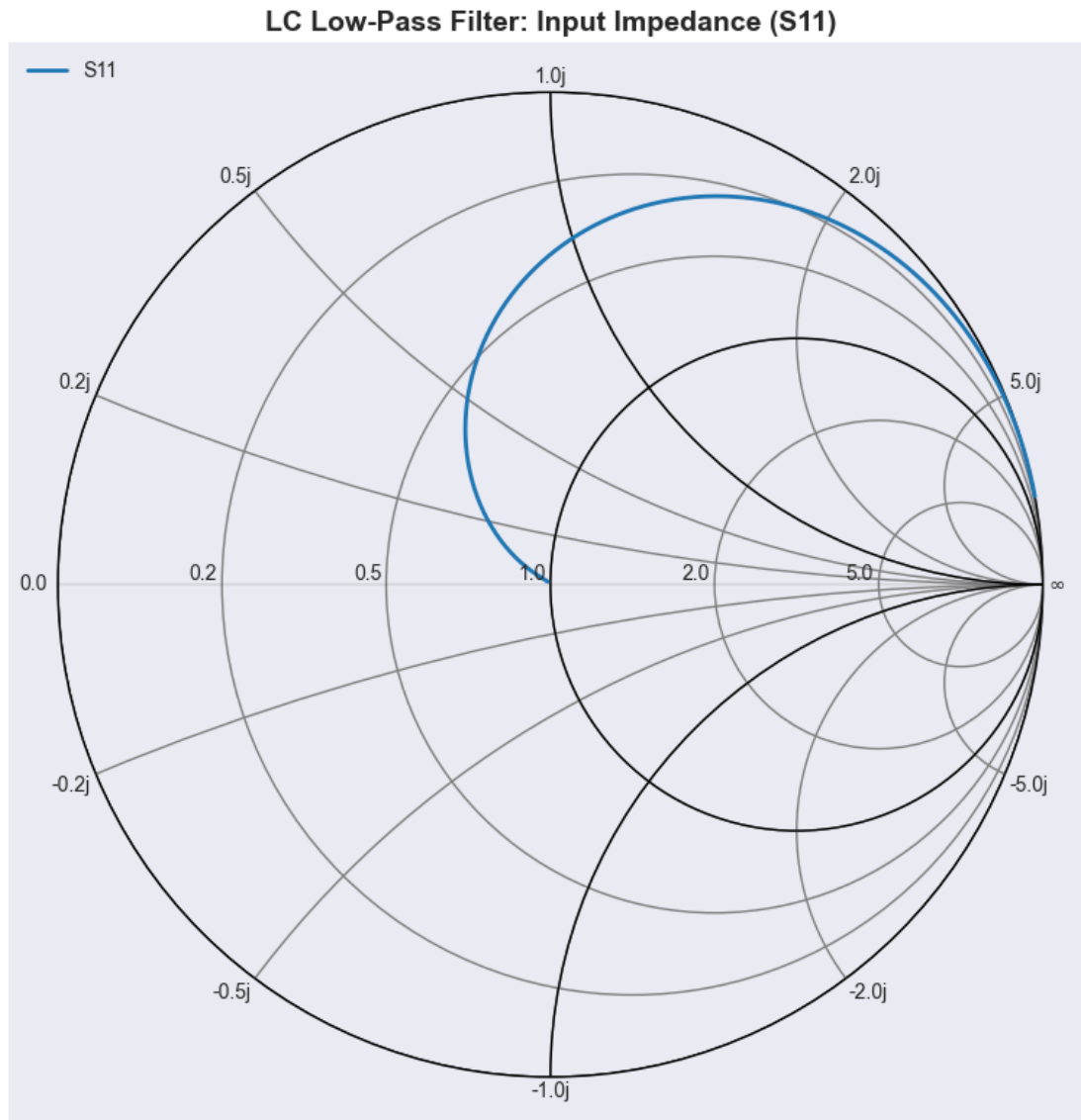
1.6.3 Smith Chart Visualization

A Smith chart provides insight into impedance matching across frequency:

```

[10]: # Plot input impedance on Smith chart
fig, ax = plt.subplots(figsize=(8, 8))
lowpass_filter.plot_s_smith(m=0, n=0, ax=ax, draw_labels=True, linewidth=2)
ax.set_title('LC Low-Pass Filter: Input Impedance (S11)', fontsize=14,
↪fontweight='bold')
plt.tight_layout()
plt.savefig('../assets/figures/lowpass_smith.png', dpi=300,
↪bbox_inches='tight')
plt.show()

```



1.7 Advanced Considerations for LC Circuit Modeling

A simple LC model is sufficient for initial analysis, but real-world filter behavior depends on more than ideal inductors and capacitors. The following considerations help ensure your simulation reflects practical RF performance.

1.7.1 Component Modeling Accuracy

At high frequencies, parasitics and layout effects can affect results. You can add parasitic elements to the model alongside the ideal components:

```
[11]: # Create inductor with parasitic capacitance and series resistance
L1_value = 8.893e-9 # H
L1_ESR = 0.5 # ohms (equivalent series resistance)
L1_Cp = 0.2e-12 # F (parallel parasitic capacitance)

# Build more realistic inductor model
L1_ideal = media.inductor(L1_value, name='L1_ideal')
L1_esr = media.resistor(L1_ESR, name='L1_ESR')
L1_parasitic_cap = media.capacitor(L1_Cp, name='L1_Cp')

# Series combination of L and ESR
L1_with_loss = L1_ideal ** L1_esr

print("Realistic inductor model created with ESR and parasitic capacitance")
```

Realistic inductor model created with ESR and parasitic capacitance

1.7.2 Topology Flexibility

Scikit-RF supports various filter topologies. Here's a T-section low-pass filter:

```
[12]: # T-section lowpass: L-C-L topology
# Two series inductors with shunt capacitor in middle

L2 = media.inductor(8.893e-9, name='L2') # Second inductor

connections_t_section = [
    [(port1, 0), (L1, 0)], # Input to first inductor
    [(L1, 1), (C1, 0), (L2, 0)], # Junction: L1, C1, L2
    [(C1, 1), (gnd, 0)], # Capacitor to ground
    [(L2, 1), (port2, 0)] # Second inductor to output
]

circuit_t = rf.Circuit(connections_t_section)
t_section_filter = circuit_t.network

print("T-section low-pass filter created")
```

T-section low-pass filter created

1.7.3 Comparison of Filter Topologies

```
[13]: # Compare L-section vs T-section
fig, ax = plt.subplots(figsize=(10, 6))

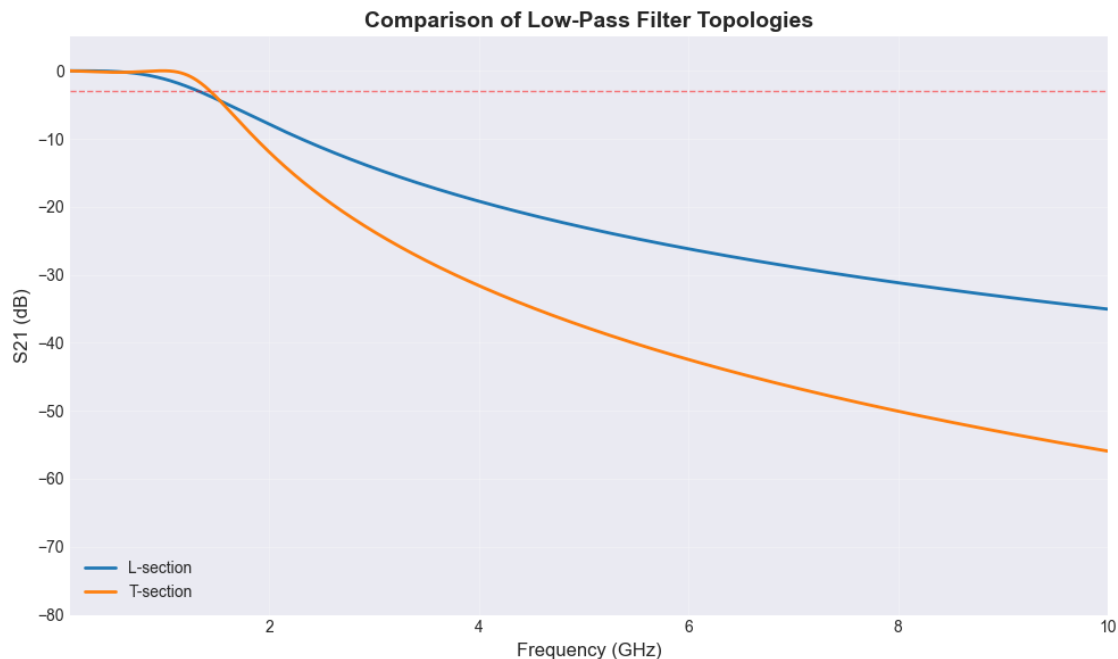
lowpass_filter.plot_s_db(m=1, n=0, ax=ax, label='L-section', linewidth=2)
t_section_filter.plot_s_db(m=1, n=0, ax=ax, label='T-section', linewidth=2)
```

```

ax.set_xlabel('Frequency (GHz)', fontsize=12)
ax.set_ylabel('S21 (dB)', fontsize=12)
ax.set_title('Comparison of Low-Pass Filter Topologies', fontsize=14,
            fontweight='bold')
ax.axhline(-3, color='red', linestyle='--', linewidth=1, alpha=0.5)
ax.grid(True, alpha=0.3)
ax.legend()
ax.set_ylim(-80, 5)

plt.tight_layout()
plt.savefig('../assets/figures/topology_comparison.png', dpi=300,
            bbox_inches='tight')
plt.show()

```



1.8 Tolerance Analysis Using Monte Carlo Simulation

Real components have manufacturing tolerances that affect filter performance. Monte Carlo analysis quantifies the statistical variation in filter response due to component tolerances.

1.8.1 Tolerance Model

We'll model typical component tolerances: - Inductors: $\pm 5\%$ tolerance - Capacitors: $\pm 10\%$ tolerance

Components typically follow a normal distribution within their tolerance range.

```

[14]: def monte_carlo_filter_analysis(nominal_L, nominal_C, L_tolerance, C_tolerance,
↳ n_iterations=100):
    """
    Perform Monte Carlo analysis on LC filter with component tolerances.

    Parameters
    -----
    nominal_L : float
        Nominal inductance in H
    nominal_C : float
        Nominal capacitance in F
    L_tolerance : float
        Inductor tolerance as fraction (e.g., 0.05 for 5%)
    C_tolerance : float
        Capacitor tolerance as fraction (e.g., 0.10 for 10%)
    n_iterations : int
        Number of Monte Carlo iterations

    Returns
    -----
    s21_results : array
        Array of S21 responses for all iterations
    cutoff_frequencies : array
        Array of -3dB cutoff frequencies
    """
    s21_results = []
    cutoff_frequencies = []

    # Use 3-sigma approach: tolerance range represents ±3
    L_sigma = nominal_L * L_tolerance / 3
    C_sigma = nominal_C * C_tolerance / 3

    for i in range(n_iterations):
        # Sample component values from normal distribution
        L_actual = np.random.normal(nominal_L, L_sigma)
        C_actual = np.random.normal(nominal_C, C_sigma)

        # Ensure positive values
        L_actual = abs(L_actual)
        C_actual = abs(C_actual)

        # Build filter with actual component values
        L_mc = media.inductor(L_actual, name=f'L_mc_{i}')
        C_mc = media.capacitor(C_actual, name=f'C_mc_{i}')

        connections_mc = [
            [(port1, 0), (L_mc, 0)],

```

```

        [(L_mc, 1), (C_mc, 0), (port2, 0)],
        [(C_mc, 1), (gnd, 0)]
    ]

    circuit_mc = rf.Circuit(connections_mc)
    filter_mc = circuit_mc.network

    # Extract S21
    s21_db = filter_mc.s_db[:, 1, 0]
    s21_results.append(s21_db)

    # Find -3 dB cutoff
    idx_3db = np.argmin(np.abs(s21_db + 3))
    fc = filter_mc.frequency.f[idx_3db]
    cutoff_frequencies.append(fc)

    return np.array(s21_results), np.array(cutoff_frequencies)

print("Monte Carlo analysis function defined")

```

Monte Carlo analysis function defined

1.8.2 Running the Monte Carlo Simulation

```

[15]: # Run Monte Carlo simulation
print("Running Monte Carlo simulation with 200 iterations...")
s21_mc, fc_mc = monte_carlo_filter_analysis(
    nominal_L=8.893e-9,
    nominal_C=3.222e-12,
    L_tolerance=0.05, # 5%
    C_tolerance=0.10, # 10%
    n_iterations=200
)
print("Monte Carlo simulation complete")

```

Running Monte Carlo simulation with 200 iterations...

Monte Carlo simulation complete

1.8.3 Visualizing Tolerance Effects

```

[16]: # Plot Monte Carlo results
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Plot all S21 traces
for s21_trace in s21_mc:
    ax1.plot(frequencies/1e9, s21_trace, alpha=0.1, color='blue', linewidth=0.5)

```

```

# Plot nominal response
ax1.plot(frequencies/1e9, lowpass_filter.s_db[:, 1, 0],
         color='red', linewidth=2.5, label='Nominal', zorder=10)

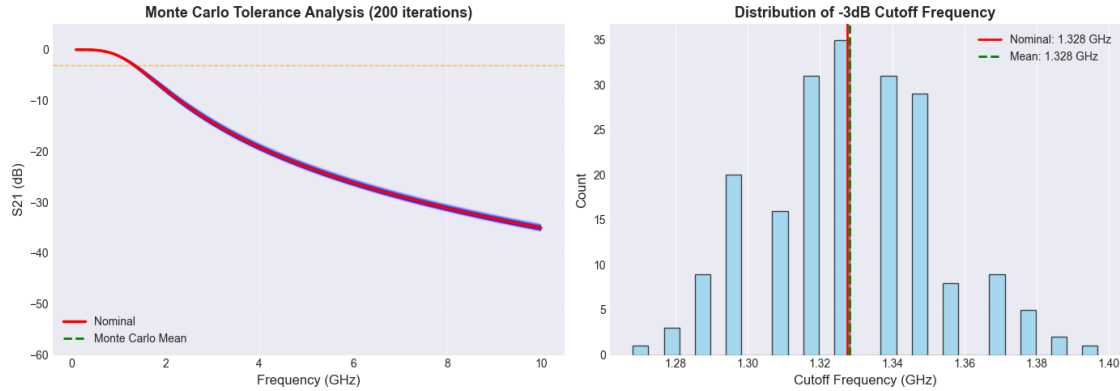
# Plot mean and  $\pm 3$  bounds
s21_mean = np.mean(s21_mc, axis=0)
s21_std = np.std(s21_mc, axis=0)
ax1.plot(frequencies/1e9, s21_mean, color='green', linewidth=2,
         label='Monte Carlo Mean', linestyle='--', zorder=9)

ax1.axhline(-3, color='orange', linestyle='--', linewidth=1, alpha=0.7)
ax1.set_xlabel('Frequency (GHz)', fontsize=12)
ax1.set_ylabel('S21 (dB)', fontsize=12)
ax1.set_title('Monte Carlo Tolerance Analysis (200 iterations)', fontsize=13,
             fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)
ax1.set_ylim(-60, 5)

# Histogram of cutoff frequencies
ax2.hist(fc_mc/1e9, bins=30, color='skyblue', edgecolor='black', alpha=0.7)
ax2.axvline(fc_actual/1e9, color='red', linewidth=2, label=f'Nominal: {fc_actual/1e9:.3f} GHz')
ax2.axvline(np.mean(fc_mc)/1e9, color='green', linewidth=2, linestyle='--',
            label=f'Mean: {np.mean(fc_mc)/1e9:.3f} GHz')
ax2.set_xlabel('Cutoff Frequency (GHz)', fontsize=12)
ax2.set_ylabel('Count', fontsize=12)
ax2.set_title('Distribution of -3dB Cutoff Frequency', fontsize=13,
             fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('../assets/figures/monte_carlo_analysis.png', dpi=300,
            bbox_inches='tight')
plt.show()

```



1.8.4 Statistical Summary

```
[17]: # Calculate statistics
fc_mean = np.mean(fc_mc)
fc_std = np.std(fc_mc)
fc_min = np.min(fc_mc)
fc_max = np.max(fc_mc)

print("=" * 60)
print("MONTE CARLO TOLERANCE ANALYSIS SUMMARY")
print("=" * 60)
print(f"Component Tolerances:")
print(f"  Inductor: ±5%")
print(f"  Capacitor: ±10%")
print(f"\nCutoff Frequency Statistics (n=200):")
print(f"  Nominal:      {fc_actual/1e9:.4f} GHz")
print(f"  Mean:         {fc_mean/1e9:.4f} GHz")
print(f"  Std Dev:      {fc_std/1e6:.2f} MHz ({100*fc_std/fc_mean:.2f}%)")
print(f"  Min:          {fc_min/1e9:.4f} GHz ({100*(fc_min-fc_actual)/fc_actual:↵+.2f}%)")
print(f"  Max:          {fc_max/1e9:.4f} GHz ({100*(fc_max-fc_actual)/fc_actual:↵+.2f}%)")
print(f"  Range:        {(fc_max-fc_min)/1e6:.2f} MHz")
print(f"\n3-Sigma Bounds:")
print(f"  Lower:        {(fc_mean - 3*fc_std)/1e9:.4f} GHz")
print(f"  Upper:        {(fc_mean + 3*fc_std)/1e9:.4f} GHz")
print("=" * 60)
```

```
=====
MONTE CARLO TOLERANCE ANALYSIS SUMMARY
=====
```

```
Component Tolerances:
  Inductor: ±5%
```

Capacitor: $\pm 10\%$

Cutoff Frequency Statistics (n=200):

Nominal:	1.3276 GHz
Mean:	1.3282 GHz
Std Dev:	23.62 MHz (1.78%)
Min:	1.2682 GHz (-4.47%)
Max:	1.3969 GHz (+5.22%)
Range:	128.70 MHz

3-Sigma Bounds:

Lower:	1.2574 GHz
Upper:	1.3991 GHz

=====

1.9 Conclusions

This whitepaper has demonstrated how Scikit-RF can be used to construct high-fidelity LC filter models suitable for RF analysis and early-stage design exploration. Key takeaways include:

1.9.1 Core Capabilities

1. **Circuit Construction:** Scikit-RF provides an intuitive framework for building RF circuits using the `Circuit` class with connection lists that explicitly define circuit topology.
2. **Media Foundation:** The `DefinedGammaZ0` media class enables the creation of lumped elements with well-defined characteristic impedance and propagation constants.
3. **S-Parameter Analysis:** Built-in visualization tools make it straightforward to analyze filter performance through S-parameters, Smith charts, and other network representations.

1.9.2 Practical Insights

4. **Component Tolerances Matter:** Monte Carlo analysis reveals that typical component tolerances ($\pm 5\%$ for inductors, $\pm 10\%$ for capacitors) can produce significant variation in cutoff frequency. Designers must account for this variation in their specifications.
5. **Topology Comparison:** Different filter topologies (L-section, T-section, π -section) offer different trade-offs between insertion loss, rolloff rate, and component count. Scikit-RF makes it easy to compare these alternatives.
6. **Parasitics and Real-World Effects:** While ideal LC models provide good first-order understanding, adding parasitic resistance and capacitance improves accuracy at higher frequencies.

1.9.3 Design Workflow

The techniques presented here support a systematic design workflow:

1. Define requirements (cutoff frequency, impedance, insertion loss)
2. Select appropriate filter topology

3. Calculate initial component values using filter theory
4. Model the filter in Scikit-RF
5. Optimize component values to meet specifications
6. Perform tolerance analysis to understand manufacturing sensitivity
7. Add parasitic models for high-frequency validation
8. Graduate to full EM simulation when layout effects become important

1.9.4 Limitations and Next Steps

While Scikit-RF excels at lumped-element analysis, designers should recognize its limitations:

- **No physical layout:** Scikit-RF does not model trace lengths, coupling, or radiation
- **Ideal connections:** All junctions are assumed to be ideal with no parasitic inductance
- **Component models:** Real components have frequency-dependent behavior not captured by simple L, C, R models

For production designs, engineers should: - Validate Scikit-RF results with EM simulation (HFSS, CST, Sonnet) - Use manufacturer component models when available - Prototype and measure to verify simulation accuracy

1.9.5 Final Thoughts

Scikit-RF modeling is a powerful way to explore RF filter concepts and iterate quickly on design alternatives. By understanding how to define frequency ranges, create lumped elements, and build network connections, engineers can adapt these methods to a wide variety of RF topologies including matching networks, bias tees, and multi-stage amplifiers.

The combination of rapid simulation, tolerance analysis, and publication-quality visualization makes Scikit-RF an essential tool in the RF engineer's toolkit for the early stages of filter design.

1.10 References

1. **Scikit-RF Documentation**
<https://scikit-rf.readthedocs.io/>
2. **Scikit-RF GitHub Repository**
<https://github.com/scikit-rf/scikit-rf>
3. **Pozar, D. M.** (2011). *Microwave Engineering* (4th ed.). Wiley.
4. **Hong, J. S., & Lancaster, M. J.** (2001). *Microstrip Filters for RF/Microwave Applications*. Wiley.
5. **Zverev, A. I.** (1967). *Handbook of Filter Synthesis*. Wiley.
6. **Scikit-RF Circuit Tutorial**
<https://scikit-rf.readthedocs.io/en/latest/tutorials/Circuit.html>
7. **Scikit-RF Media Classes**
<https://scikit-rf.readthedocs.io/en/latest/api/media/index.html>
8. **Williams, A. B., & Taylor, F. J.** (2006). *Electronic Filter Design Handbook* (4th ed.). McGraw-Hill.

1.10.1 About the Author

Peter Matthews specializes in RF and microwave circuit design with applications in defense, aerospace, and precision sensing.

1.10.2 Contact

For advanced filter design services and manufacturing solutions, visit:
<https://www.knowles.com/about-knowles/about/doing-business-with-knowles>